

Rapport de validation Groupe 5



Client : Isabelle Ferrane

Membre du groupe : Sébastien Delhem, Antonin Gerain, Lucas Haegel, Adrien Nosel, Ny Aina Raharitsifa

Sommaire

I - Introduction.....	3
1.1 - Remise en contexte du projet :.....	3
1.2 - Enjeux du projet.....	3
II - Architecture logicielle.....	4
2.1 - Interface utilisateur.....	4
2.2 - Traitement des commandes vocales.....	5
2.3 - Traitement des commandes textuelles.....	5
2.4 - Traitement des images.....	6
III - Organisation et communication interne :.....	7
3.1 - Gestion du temps de travail.....	7
3.2 - Outils de communication et gestion de projet.....	9
IV - Bilan individuel :.....	10
Sébastien Delhem :.....	10
Antonin Gerain :.....	11
Lucas Haegel :.....	12
Adrien Nosel :.....	12
Ny Aina Raharitsifa :.....	16
V - Cas d'erreurs et amélioration.....	17

I - Introduction

1.1 - Remise en contexte du projet :

Ce projet a pour objectif de créer une plateforme mobile autonome pouvant évoluer dans un environnement contrôlé. Le PFR1 se focalise sur le traitement de requêtes vocales et textuelles ainsi que sur l'analyse d'image. Le tout sera simulé sur une interface et pourra être visualisé et testé par un utilisateur qui pourra interagir avec la simulation.

1.2 - Enjeux du projet

L'enjeu et le défi pour notre équipe a été la coordination de groupe car la plupart des membres avaient déjà des bases de connaissances en ce qui concerne le codage mais avait peu d'expérience dans la réalisation d'un travail de groupe et plus généralement à un projet de la lecture du cahier des charges à la livraison au client.

L'effort a été concentré sur notre capacité à communiquer et à faire fonctionner un projet de groupe en passant d'un travail individuel à un travail collectif.

II - Architecture logicielle

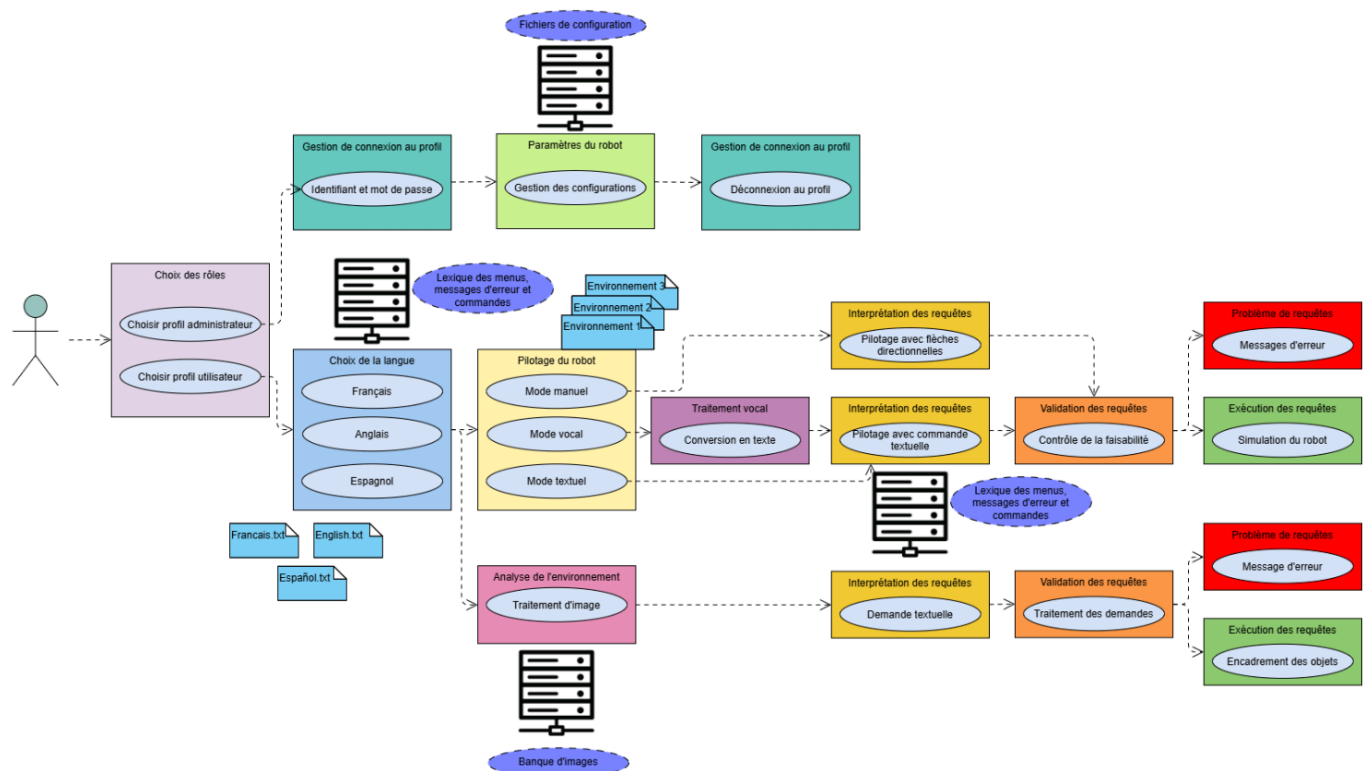


Diagramme des cas d'utilisations du robot

2.1 - Interface utilisateur

Accessible depuis un terminal de commande, l'utilisateur peut naviguer au sein des différents menu en entrant les numéros correspondant à l'endroit de l'environnement où il souhaite se rendre.

Une fois le programme lancé, l'utilisateur doit dans un premier temps sélectionner la langue de l'interface souhaitée, français ou anglais, puis choisir entre le menu utilisateur ou administrateur.

Le menu utilisateur permet d'utiliser les fonctionnalités du robot : son pilotage et le traitement d'image. Le pilotage du robot peut être effectué avec des requêtes textuelles saisies au clavier ou via des instructions vocales. Ces commandes permettent de contrôler le robot dans un environnement sélectionné. Dans le PFR1 l'évolution du robot s'effectue via une simulation affichée à l'écran une fois les requêtes envoyées.

Lorsque l'utilisateur sélectionne le traitement d'image, le programme de traitement s'effectue en affichant le bit de quantification utilisé. Les images traitées sont visualisables dans les fichiers du projet avec leurs matrices respectives.

Le menu administrateur permet de modifier la configuration du traitement d'image : le bit de quantification. Pour y accéder, l'utilisateur doit saisir un identifiant ("admin") et un mot de passe ("robot").

2.2 - Traitement des commandes vocales

Cette fonctionnalité est liée à la fonctionnalité de traitement de texte car l'utilisateur peut décider d'utiliser la reconnaissance vocale mais celle-ci appelle la fonctionnalité de requête text pour commander le système.

Cette fonctionnalité traite plusieurs langues, notamment de Français et l'Anglais, utilise une synthèse vocale pour donner un feedback en temps réel à l'utilisateur sur la commande reconnue.

2.3 - Traitement des commandes textuelles

Les requêtes textuelles passent par 3 étapes avant d'arriver à la partie simulation:

- l'analyse lexicale;
- l'interprétation en commandes turtles;
- l'export vers un fichier .txt.

L'analyse lexicale a pour but de découper la phrase transmise par l'utilisateur en plusieurs mots. Elle supprime certaines ponctuations ou caractères pouvant rendre difficile la reconnaissance du mot (ex: "droite, " et "droite"). Elle sépare aussi les nombres des unités si ces derniers sont collés (ex: 5m doit être géré comme 5 m). Elle reconnaît ensuite le mot à partir du lexique chargé depuis le choix de la langue au lancement du logiciel: si l'utilisateur a choisi Français comme langue, un lexique en français contenant les mots clés utiles pour la commande sera chargé. Ces mots sont catégorisés en plusieurs types: Actions, Directions, Unités, Nombres, Objets, Couleurs, Mots de liaison, Actions spéciales (ex: demi-tour), Négation, Inconnus.

Les mots typés sont ensuite envoyés à la partie interprétation qui les transformera en commande turtle. Une action génère une commande à laquelle peut s'ajouter les paramètres d'angle, de direction ou de distance. Lorsque ces paramètres sont absents des valeurs par défaut sont générés afin que le robot fasse quand même quelque chose (ex: tourne -> left 90). Certaines expressions sont traitées comme des actions spéciales et sont ainsi interprétées comme une composition d'action simple. La négation distingue 2 cas: une négation globale placée avant l'action qui annule la commande (ex: ne tourne pas) et une négation locale placée après l'action qui annulera le paramètre qui le succède (ex: tourne pas à gauche mais à droite).

Les commandes générées sont exportées dans un fichier texte, une commande par ligne, dans l'ordre d'exécution. Si aucune commande n'a été générée, la fonction qui demande à écrire le texte retourne un 0 qui va afficher sur l'écran un message d'erreur.

Une fois la commande textuelle entrée par l'utilisateur analysé, et le fichier de configuration des commandes à exécuter mis à jour, le programme contenu dans [ReadCmd.py](#) va venir lire dans ce fichier de configuration, puis, venir lancer dynamiquement les instructions qu'il y lira et fera se déplacer le robot dans l'interface de simulation préalablement générée.

2.4 - Traitement des images

La partie traitement de l'image pour le PFR1 a consisté à transformer une image RGB classique en une version quantifiée, puis à construire un histogramme de ces valeurs quantifiées. Cette démarche nous a permis de réduire la complexité du problème et de rendre les images comparables numériquement. C'est cette base qui prépare la suite du projet, même si les recherches sur les formes et la reconnaissance d'objets n'ont pas encore été implémentées à ce stade.

III - Organisation et communication interne :

3.1 - Gestion du temps de travail

Diagramme de gantt prévisionnel :

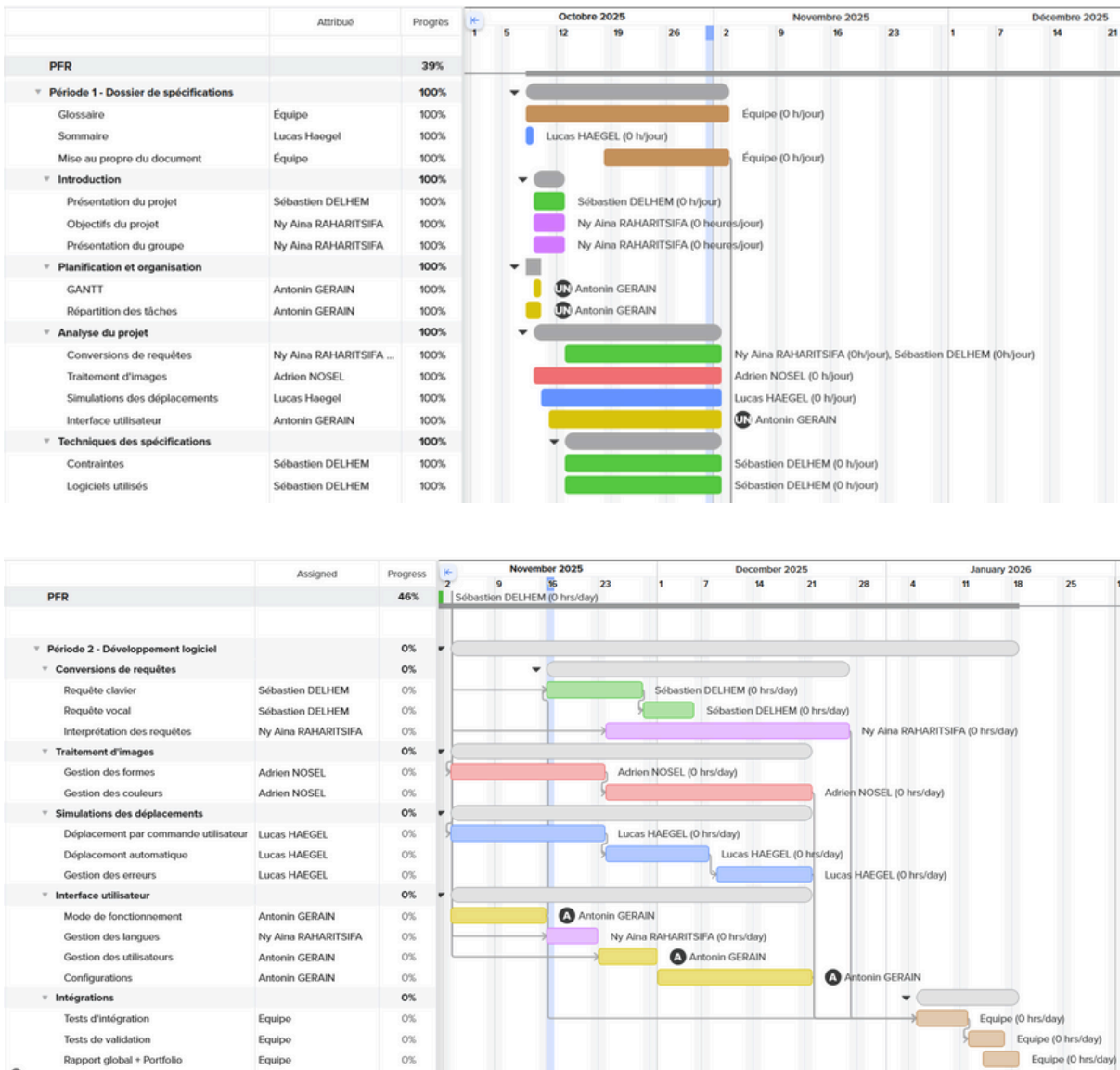
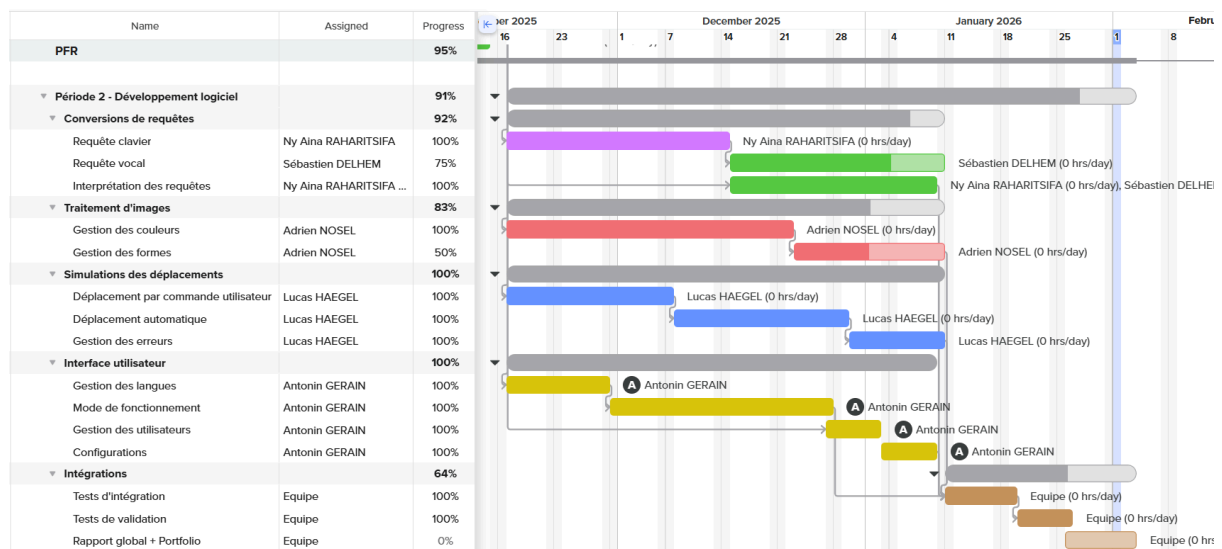
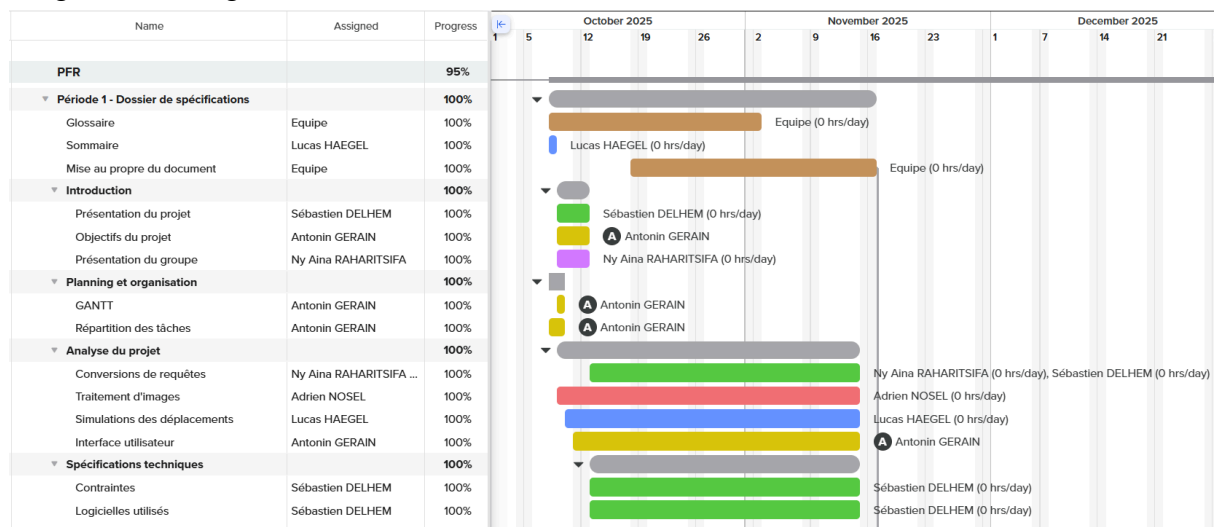


Diagramme de gantt réel



Nous avons eu beaucoup de mal à nous tenir aux délais prévus initialement pour l'ensemble des tâches nécessaires à la bonne réalisation du projet. Suite à un retard de deux semaines dû au rendu du dossier de spécification et à l'arrivée d'une grosse période de travail suite aux études, nous n'avons pas pu rattraper le retard emmagasiné au lancement de la période de codage en novembre. Suite à ça, nous avons en janvier revue complètement l'organisation de la réalisation des tâches pour ce concentrer sur les tâches critiques afin de pouvoir terminer à temps et avec une bonne qualité ce projet.

De plus, nous avons réussi à terminer dans les délais toutes les tâches que nous nous étions fixées début janvier pour le rendu du projet et cela sans qu'une tâche ne retarde la conception d'une autre.

Finalement, nous n'avons pas réussi à pousser certaines fonctionnalités du système aussi loin que prévu initialement, comme par exemple l'interface vocale qui n'a pas pu être mise en fonction avec l'analyse textuelle pour faire se déplacer le robot.

3.2 - Outils de communication et gestion de projet

Pour la communication entre le groupe de projet et le client, nous avons essentiellement utilisé la messagerie intégrée à l'université et l'espace moodle pour les comptes rendus. La fréquence des réunions était d'une réunion toutes les une à deux semaines pour que le client ait un suivi régulier de l'avancement du projet et lui demander des précisions pour la conception.

Pour la communication en interne, nous avons utilisé Discord, Google Drive et GitHub pour partager notre travail et faire un suivi de l'avancement du projet.

Pour ce qui est de la conception du diagramme de GANTT, nous avons utilisé le site internet TeamGANTT.

IV - Bilan individuel :

Sébastien Delhem :

Ma contribution dans le projet fil rouge a été d'établir la communication entre le client et les membres du groupe, même si cette tâche n'a pas toujours été respectée dans les délais.

J'ai aussi pris en charge au côté de Ny Aina la fonctionnalité de requête texte mais par manque de temps et de compétences, j'ai pris la décision de ne travailler uniquement sur la fonctionnalité de la reconnaissance vocale et de son intégration avec le reste du code.

Durant la validation, pour ne pas empiéter sur le temps de parole de mes camarades, j'ai omis de parler de certains détails concernant l'utilisation de la fonctionnalité reconnaissance vocale, ce qui fonctionne et ce qui ne fonctionne pas.

Cette fonctionnalité permet d'entendre l'utilisateur, de traduire ces paroles en texte et d'envoyer ce test à la fonctionnalité de requête commande.

Cette fonctionnalité prend en compte différentes langues, notamment le Français et l'Anglais, une synthèse vocale permet de faire un feedback auditif pour savoir si la commande envoyée est bien la commande que l'utilisateur a demandé.

Tous les mots de la langue française (et anglaise) sont compris et interprété par la fonctionnalité de reconnaissance vocale, ce qui fait que nous pouvons dire la phrase "avance de rouge mètres et tourne à bleu" sans que cela ne pose de problème.

J'ai entrepris de faire en sorte que cette requête soit transférer dans un document .txt pour que la fonctionnalité de requête text requête commande puisse la lire et commander le système mais après de multiples bug, j'ai pris la décision de ne pas intégrer cette fonction pour ne pas gâcher la présentation.

Toutefois, les problèmes rencontrés ont été communiqués à l'équipe pour que la fonction soit disponible prochainement.

Idem pour l'appel de la simulation, étant donné que celle- ci est appelée après la requête commande.

Antonin Gerain :

Durant le PFR1, je me suis occupé de l'interface utilisateur ainsi que de l'intégration de toutes les parties du projet. Au début du projet, après la validation du dossier de spécification, j'ai eu des difficultés pour commencer à développer l'interface car je ne savais pas par où commencer ni comment l'interface devait se présenter. Après les différentes réunions et discussions avec les membres du groupe, j'ai pu mieux comprendre ce qui était attendu et j'ai ainsi pu me lancer dans la conception. Le développement du code, programmé en langage C, pour l'affichage et la navigation dans les menus ne m'a pas posé de grandes difficultés. L'intégration de toutes les parties s'est bien réalisée car j'ai pu créer une architecture de dossiers facilitant la portabilité des programmes.

Quelques difficultés se sont présentées lors du développement du code. La première a été d'intégrer la traduction des menus en anglais. J'ai dû créer deux fichiers textes correspondant aux deux lexiques (français et anglais) et les intégrer dans le programme. Pour cela, je devais gérer la notion de traitement de fichiers en C et stocker les informations de texte dans une structure comportant deux tableaux (cf. lang.c). Ces notions étant survolées en cours, j'ai dû m'informer sur différents forums et discuter avec d'autres groupes.

La deuxième difficulté a été de rendre le logiciel robuste grâce au contrôle de la saisie au clavier lors de la navigation dans les menus. Pour cela, j'ai dû aborder la notion de buffer, qui a été peu vu pendant les cours de programmation en C. Comme pour le lexique, je me suis informé sur des forums et j'ai effectué des recherches afin de mieux comprendre ces notions. Grâce à cela, j'ai pu élaborer une fonction permettant de vérifier si la saisie est correcte et d'envoyer un message d'erreur si ce n'est pas le cas.

La troisième grande difficulté a été la première intégration dans l'interface : le mode requête textuelle. Dans un premier temps, j'ai dû relier le choix de la langue de l'interface avec celui du lexique des requêtes afin de rendre le logiciel cohérent. Cette partie a bien fonctionné, sauf que lorsque l'on voulait envoyer une requête, celle-ci n'était pas prise en compte, comme si le lexique ne se chargeait pas. Finalement, j'ai compris que lorsque l'on sélectionnait le mode textuel, le programme que j'appelais était le main des requêtes alors qu'un main était déjà en exécution : celui de l'interface. Après la correction de cette erreur, le programme fonctionnait normalement.

Une de mes responsabilités lors de ce projet a été d'être le chef de projet. Ce rôle m'a amené à communiquer régulièrement avec tous les membres du groupe pour avoir un retour de leur avancement sur leurs parties respectives et ainsi m'organiser pour faciliter l'intégration. Ce rôle m'a permis de développer mes compétences en communication et en organisation.

Cette première partie du projet m'a permis d'être plus à l'aise pour coder en langage C et d'approfondir mes connaissances, notamment sur la structuration d'un programme et l'organisation du code en plusieurs modules. Elle m'a également permis de mieux comprendre l'importance d'une architecture claire pour faciliter l'intégration des différentes parties du logiciel.

Ce projet m'a aussi aidé à gagner en autonomie dans l'utilisation des commandes Unix, que ce soit pour compiler les programmes, naviguer dans les dossiers ou gérer les fichiers du projet. Cette phase m'a permis de mieux appréhender les difficultés liées au développement logiciel et d'améliorer ma méthodologie de travail.

Lucas Haegel :

Dans le cadre du PFR1, je me suis occupé de la partie simulation et exécution des commandes. Dans cette partie, l'objectif fondamental été d'offrir à l'utilisateur une visualisation simple et compréhensible de l'environnement. Dans cet objectif j'ai configuré deux environnements simples et épurés avec un tracé des déplacements effectués du robot.

Les commandes textuelles fournies par le fichier configuration sont récupérées ligne par ligne et exécutées au fur et à mesure pour faire se déplacer le robot.

La gestion des obstacles se fait par placement du centre de l'objet puis par une création de la forme. Cet algorithme reste proche de la réalité car il permet de connaître la position des objets pour pouvoir les éviter si besoin.

Futur du projet :

Pour la suite du projet, il faudra permettre au programme de boucler sur l'exécution des commandes de déplacements pour permettre par l'utilisateur de déplacer le robot sans avoir à re-générer l'environnement.

Adrien Nosel :

Au cours de mon projet, j'ai cherché à construire une démarche claire et progressive, plutôt que de viser immédiatement une solution complète et parfaite. Mon objectif principal était de mettre en place une chaîne de traitement d'images capable d'identifier des balles de couleur dans des images en couleur, puis d'en extraire des informations de plus en plus structurées. Dès le début, j'ai réalisé que le terme "reconnaître une balle" pouvait avoir plusieurs sens différents, et que mélanger tous ces objectifs dès le départ pourrait rendre le projet difficile à comprendre et à valider. J'ai donc décidé de me concentrer sur une étape de base : apprendre à simplifier les images pour pouvoir les analyser correctement par la suite. J'ai commencé par essayer de comprendre comment décrire une image en utilisant ses pixels, de manière quantitative et comparable d'une image à une autre, plutôt que de chercher immédiatement à reconnaître des objets ou des formes.

J'ai commencé à me demander si cela valait la peine d'ajouter un filtre moyennneur ou médian après la quantification. Mais comme la mise en place des fonctions de quantification

a pris plus de temps que prévu et que les résultats des fonctions étaient meilleurs que ce que j'attendais en termes de qualité, j'ai réévalué l'utilité de ces traitements supplémentaires et j'ai décidé de ne pas en implémenter dans ma solution.

La première chose à considérer est l'histogramme de couleurs. Cela signifie compter le nombre de pixels qui ont une intensité spécifique. Au lieu de voir l'image comme une scène, je la vois comme une répartition de valeurs. Je veux savoir combien de pixels ont une intensité de 0, combien ont une intensité de 1, et ainsi de suite jusqu'à 255, puisque nous travaillons avec des intensités sur 8 bits. Cette représentation a un avantage direct : elle transforme une image en une signature numérique, qui peut être analysée, comparée et utilisée pour des traitements automatiques.

Cependant, travailler directement avec toutes les valeurs possibles sur 8 bits pour chaque composante couleur peut vite devenir lourd, surtout si l'objectif est de comparer des images ou de faire des calculs répétés. C'est pour cette raison que nous avons une étape de prétraitement essentielle : la quantification. Le principe est de limiter l'information retenue sur chaque composante couleur (R, G, B) en ne conservant que les bits de poids forts. L'objectif est clair : réduire le nombre de cas possibles et donc réduire les calculs, tout en gardant une information suffisante et pertinente pour une analyse globale.

J'ai appliqué cette quantification en retenant uniquement les n bits de poids forts, ce qui revient à ne conserver que les premières puissances de 2 dans la représentation binaire de l'intensité. Une fois cette étape comprise, j'ai organisé ces informations sous la forme d'un codage compact à 6 bits, en choisissant une structure explicite : R1 R2 | G1 G2 | B1 B2. Cette organisation est importante parce qu'elle définit une convention de lecture : les deux premiers bits proviennent du rouge, les deux suivants du vert, et les deux derniers du bleu. Grâce à ce regroupement, un pixel quantifié n'est plus décrit par trois intensités sur 0–255, mais par un seul nombre entier sur 6 bits.

Avec l'exemple (255, 128, 64), les bits obtenus forment la séquence $(111001)_b$. Cette séquence représente la valeur décimale 57. Le passage d'un triplet de couleur, composé de rouge, vert et bleu, vers une valeur unique est un élément essentiel de ma démarche. En effet, cela simplifie considérablement la suite des opérations : au lieu d'avoir affaire à un espace de couleurs immense et complexe, je me retrouve avec un nombre limité de valeurs possibles, ce qui facilite grandement le traitement. En quantifiant sur 2 bits par composante, j'obtiens des pixels dont la valeur est comprise entre 0 et 63, donc 64 couleurs possibles.

La prochaine étape consiste à définir clairement la procédure à appliquer à l'ensemble d'une image : pour chaque pixel de l'image, je réalise le prétraitement de quantification, puis j'incrmente un compteur qui correspond à la valeur obtenue pour le pixel. À la fin, j'obtiens un histogramme à 64 cases (pour $\text{nb_bits} = 2$), où chaque case représente le nombre de pixels ayant une valeur quantifiée donnée.

Deux images, même si elles ne sont pas identiques pixel par pixel, peuvent être rapprochées si leurs distributions de valeurs à quantifiées se ressemblent. Inversement, si leurs histogrammes sont très différents, cela indique une différence globale de contenu colorimétrique. J'ai identifié que cette comparabilité ouvre la porte à plusieurs méthodes de mesure : on peut calculer une différence entre histogrammes, ou une intersection, selon ce que l'on cherche à évaluer.

Ce travail sur la quantification et les histogrammes m'a servi de socle méthodologique. Plutôt que de me baser sur une impression visuelle, j'ai construit une manière de mesurer l'image, de la résumer, et de comparer objectivement deux résultats. À ce stade, je n'étais pas encore dans une logique de reconnaissance de formes ou de segmentation d'objets : l'objectif était d'obtenir une représentation robuste et simple, utilisable comme entrée d'analyse. Ce socle est nécessaire avant de passer à des étapes plus avancées. Il permet d'avoir des outils pour valider les résultats. Cela signifie que l'on peut vérifier si un traitement modifie bien l'image comme prévu. De plus, cela permet de quantifier ces modifications au lieu de simplement les supposer. Le socle permet donc de valider les résultats de chaque traitement d'image.

Le futur du projet :

Je fais un pas en arrière pour mieux avancer dans mes objectifs.

- Je compte d'abord réorganiser mon histogramme pour qu'il soit plus cohérent avec la façon dont je vais l'utiliser pour comparer mes images.
- Je vais ensuite reprendre mes recherches sur la détection de formes. J'ai fait l'erreur de vouloir aller trop vite en mélangeant la recherche de formes par des composantes dans la matrice et de couleur en utilisant mal mon histogramme.
- Je travaillerai ensuite à implémenter la solution au programme commun de mon groupe de projet.

Annexe image :

Image brut:

93	93	94	94	95	95	95	95
100	99	100	98	98	97	97	97
180	182	185	187	196	197	199	200
200	201	202	199	211	211	211	211
200	98	99	101	100	98	98	101
19	188	190	194	194	193	194	196
	201	200	202	202	200	202	202
	200	200	198	196	198	198	195
	193	194	194	193	193	192	192
	198	198	197	199	198	198	201



Image quantifiée sur 2 bits:

300	300	1					
16	16	16	16	16	16	16	16
16	16	16	16	16	16	16	16
58	58	58	58	58	58	58	58
58	58	58	58	58	58	58	58
58	58	42	42	42	58	58	58
58	62	58	58	58	58	58	58

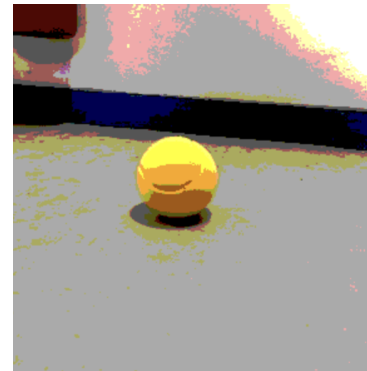


Image quantifiée sur 3 bits:

300	300	1					
128	128	128	128	128	128	128	128
192	192	192	128	128	192	192	192
282	283	283	356	356	356	356	356
429	429	429	429	429	429	429	429
429	429	429	429	429	429	429	429
429	429	429	429	429	429	429	365



Image quantifiée sur 4 bits:

300	300	1					
1296	1296	1280	1280	1296	1296	1296	1296
1280	1552	1552	1280	1280	1280	1536	1552
1553	1553	1553	1553	1569	1570	1586	1859
2986	2986	2986	2986	2986	2986	2986	3002
3259	3259	3259	3259	3259	3259	3259	3275
3259	3259	3259	3259	3259	3259	3259	3259



Ny Aina Raharitsifa :

Durant cette première partie du Projet PFR, j'ai pris en charge tout ce qui est requête texte, c'est-à-dire les commandes tapées au clavier par l'utilisateur et l'interprétation en commandes turtles.

Au début, je me suis créé un petit lexique et essayé de travailler avec ça pour au moins apprendre à lire dans un fichier et donc le traduire en commande turtle. Une fois cette étape faite, la deuxième était de le refaire mais en écrivant des phrases plus longues et des phrases composées. C'est là qu'une première grosse difficulté s'est présentée. Il s'agit de la gestion de plusieurs cas: au début il ne reconnaissait pas par exemple le mot "droite," car la virgule est collée, ou encore la requête "avance de 5m" car il ne comprenait pas que 5 et m sont deux mots à traiter.

Place ensuite à la partie interprétation qui a probablement été la partie la plus dure et complexe. En effet, pour éviter des requêtes insensées ou incomplètes, les mots d'actions doivent primer pour avoir une commande. Par exemple, si l'utilisateur écrit "gauche 63", je ne voulais pas avoir comme commandes générées "left 63" car la requête n'a pas de sens, il faut écrire à minima le mot "tourne" ou un synonyme.

Une autre grosse difficulté a été la gestion des négations. En effet au début je n'avais pas conscience que ça allait être si compliqué. Je pensais qu'il suffisait de détecter la négation autour de l'action pour annuler la commande (ex: n'avance pas) mais le langage naturel humain est bien plus complexe. Par exemple, si on écrit "ne tourne pas" ou "tourne à gauche mais pas à droite" ou "tourne pas à gauche mais à droite". Ces cas ont finalement été gérés par le placement de la négation dans la phrase.

Pour éviter de trop surcharger mon code et pour que ce soit plus lisible, j'ai décidé de séparer les parties séparations/analyses de la phrase et interprétation. Cela m'a permis aussi de me retrouver dans mon code pour le modifier assez facilement à chaque fois que j'avais des cas qui me venaient en tête ou auquel je n'avais pas pensé en amont.

Ce projet m'a permis d'énormément progresser en programmation en C, un langage auquel je n'étais pas forcément habitué avant cette année. J'ai également gagné en autonomie.

V - Cas d'erreurs et amélioration

Durant le développement du projet, nous avons rencontré plusieurs problèmes que nous n'avons pas réussi à corriger. L'un des principaux problèmes a été de relier le choix de la pièce dans laquelle évolue le robot, effectué depuis l'interface, avec le programme Python de la simulation. Nous n'avons pas réussi à transférer correctement la valeur de la variable correspondant au choix de la pièce depuis le programme c de l'interface vers celui de la simulation en python, ce qui a limité le fonctionnement complet de cette partie du projet.

Le projet présente également des points d'amélioration. Par exemple, au niveau du menu administrateur, des fonctionnalités supplémentaires pourraient être ajoutées, comme l'ajout de nouveaux fichiers de configuration. Cela permettrait l'ajout de nouveaux mots dans le lexique des requêtes ou l'intégration d'une nouvelle langue pour l'interface, rendant le logiciel plus évolutif et plus flexible.

Un point d'amélioration sur la fonctionnalité de reconnaissance vocale serait de transmettre les informations que l'utilisateur donne à l'oral, à la fonctionnalité de requête texte pour faire se déplacer le système, de la même façon, appeler le code qui gère la simulation pour un feedback visuel.